



Tutorial: Feature Store & Feast

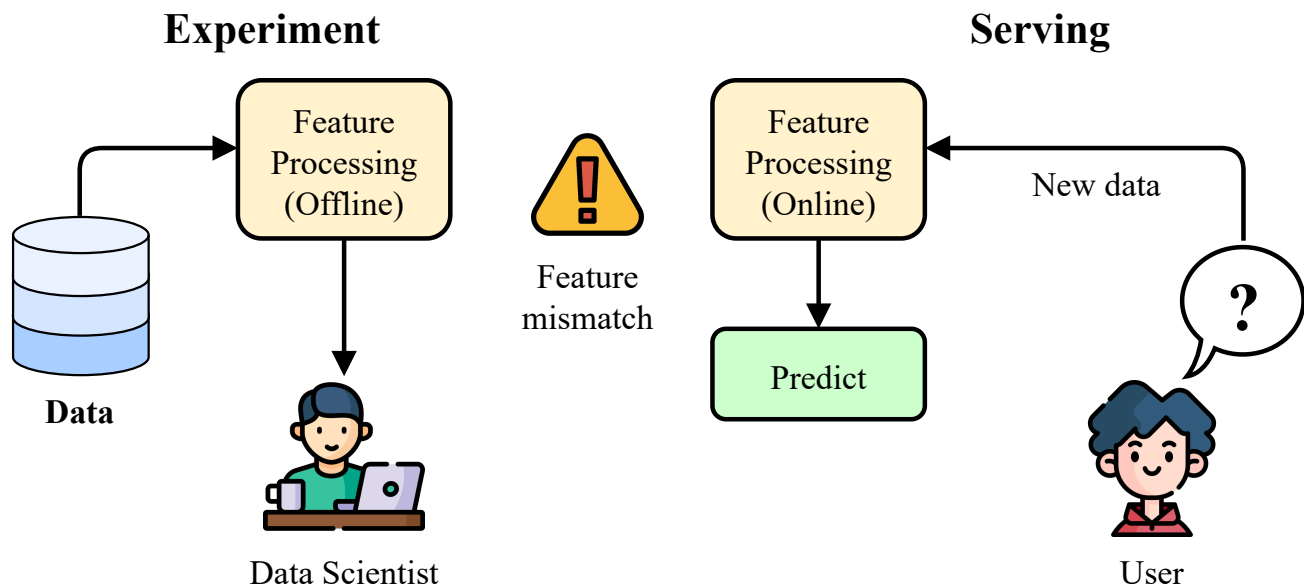
Dương Trường Bình

Dương Đình Thắng

Đình Quang Vinh

I. Dẫn nhập

Hành trình xây dựng một hệ thống Machine Learning thường khởi đầu trong môi trường thử nghiệm linh hoạt như Jupyter Notebook, nơi các nhà khoa học dữ liệu có thể nhanh chóng xử lý dữ liệu, thiết kế các **đặc trưng** phức tạp và huấn luyện mô hình đạt độ chính xác ấn tượng. Những kết quả thuận lợi ở giai đoạn này sẽ tạo tiền đề cho việc ứng dụng mô hình vào thực tế để tạo ra giá trị kinh doanh. Một ví dụ điển hình cho ứng dụng này, và cũng là bài toán xuyên suốt phần thực hành của chúng ta, là xây dựng một hệ thống có khả năng phát hiện gian lận thẻ tín dụng theo thời gian thực.



Hình 1: Sự thiếu nhất quán trong logic xử lý đặc trưng giữa môi trường thử nghiệm (trái) và vận hành (phải) dẫn đến rủi ro sai lệch, làm giảm hiệu suất của mô hình trên thực tế.

Tuy nhiên, việc chuyển đổi một mô hình từ môi trường thử nghiệm sang một hệ thống vận hành ổn định là một thách thức lớn. Vấn đề cốt lõi thường không nằm ở bản thân thuật toán, mà ở một khía cạnh phức tạp hơn: **dữ liệu và các đặc trưng được tạo ra từ dữ liệu đó**. Logic tính toán đặc trưng, vốn khá đơn giản trong môi trường batch processing khi huấn luyện, lại trở nên cực kỳ khó để tái hiện một cách nhất quán trong hệ thống real-time serving vốn đòi hỏi độ trễ rất thấp.

Mục lục

I.	Dẫn nhập	1
I.1.	Từ Machine Learning đến MLOps	6
I.2.	Những thách thức trong quản lý đặc trưng	8
II.	Tổng quan về Feature Store	9
II.1.	Vai trò và vị trí trong kiến trúc MLOps	9
II.2.	Các thành phần cốt lõi	10
II.3.	Các Feature Store phổ biến trên thị trường	13
II.4.	Feast	14
III.	Thực hành	17
IV.	Câu hỏi trắc nghiệm	26
V.	Tài liệu tham khảo	28
	Phụ lục	29

Sự thiếu nhất quán giữa hai môi trường này là nguyên nhân gây ra nhiều vấn đề nghiêm trọng, ảnh hưởng trực tiếp đến hiệu quả và độ tin cậy của sản phẩm. Ví dụ, làm thế nào để đảm bảo đặc trưng “số lượng giao dịch của người dùng trong 5 phút qua” được tính toán giống nhau cả khi huấn luyện trên dữ liệu lịch sử lẫn khi dự đoán cho một giao dịch đang diễn ra? Hoặc làm thế nào để các nhóm khác nhau có thể chia sẻ và tận dụng lại các đặc trưng đã được xây dựng, thay vì phải tốn thêm thời gian và công sức để phát triển lại từ đầu?

Để giải quyết những thách thức này và kết nối hiệu quả giữa giai đoạn nghiên cứu và giai đoạn vận hành, một thành phần hạ tầng chuyên biệt đã ra đời và trở thành phần cốt lõi trong các hệ thống MLOps (Machine Learning Operations) hiện đại: **Feature Store**. Bài viết này sẽ giới thiệu về Feature Store, bắt đầu từ vai trò và vị trí của nó trong hệ sinh thái MLOps, trình bày các chức năng chính, và cuối cùng là hướng dẫn thực hành chi tiết với Feast, một trong những Feature Store mã nguồn mở hàng đầu.

Bảng chú thích thuật ngữ

Thuật ngữ	Giải thích
MLOps	(Machine Learning Operations) Bộ quy tắc và công cụ để tự động hóa, chuẩn hóa vòng đời của hệ thống Machine Learning, từ huấn luyện đến triển khai và giám sát.
Feature Store	Hệ thống trung tâm để quản lý, lưu trữ, và cung cấp các đặc trưng một cách nhất quán cho cả việc huấn luyện và dự đoán.
Feature	(Đặc trưng) Một biến đầu vào có thể đo lường được, dùng làm tín hiệu cho mô hình học và dự đoán. Ví dụ: tuổi khách hàng, số tiền giao dịch trung bình trong 1 giờ.
Feature Vector	(Vector đặc trưng) Một mảng một chiều chứa các giá trị đặc trưng của một thực thể tại một thời điểm, được dùng làm đầu vào cho mô hình để dự đoán.
Entity	Một đối tượng kinh doanh trong thế giới thực mà các đặc trưng mô tả, đóng vai trò là khóa kết nối (join key) để tra cứu đặc trưng. Ví dụ: một khách hàng, một sản phẩm.
Training-Serving Skew	Hiện tượng hiệu suất mô hình sụt giảm trên thực tế do sự khác biệt trong logic xử lý đặc trưng giữa môi trường huấn luyện (training) và môi trường phục vụ (serving).
Model Drift	Hiện tượng hiệu suất của mô hình suy giảm theo thời gian do sự thay đổi trong mối quan hệ giữa các đặc trưng và biến mục tiêu.

Data Drift	Sự thay đổi trong phân phối thống kê của dữ liệu đầu vào giữa lúc huấn luyện và lúc phục vụ, có thể làm giảm hiệu suất mô hình.
Offline Store	Kho chứa dữ liệu lịch sử của các đặc trưng, được tối ưu cho việc truy vấn các tập dữ liệu lớn nhằm mục đích huấn luyện mô hình.
Online Store	Kho chứa chỉ lưu giá trị mới nhất của các đặc trưng, được tối ưu cho việc truy vấn với độ trễ cực thấp để phục vụ dự đoán thời gian thực.
Data Warehouse	Hệ thống kho dữ liệu tập trung, lưu trữ dữ liệu có cấu trúc đã qua xử lý, tối ưu cho việc phân tích và báo cáo.
Data Lake	Kho lưu trữ dữ liệu thô ở nhiều định dạng khác nhau (có cấu trúc, bán cấu trúc, phi cấu trúc) ở quy mô lớn.
Batch Processing	(Xử lý theo lô) Quy trình xử lý dữ liệu trên một tập hợp lớn, tĩnh, thường được áp dụng khi huấn luyện mô hình trên dữ liệu lịch sử.
Real-time Serving	(Phục vụ thời gian thực) Quy trình cung cấp dự đoán ngay lập tức khi có yêu cầu mới, đòi hỏi truy xuất dữ liệu và tính toán với độ trễ thấp.
Materialization	Quá trình tính toán và nạp các giá trị đặc trưng từ Offline Store vào Online Store để phục vụ cho việc dự đoán thời gian thực.
Point-in-time Correctness	(Tính chính xác tại thời điểm) Tính năng đảm bảo khi tạo dữ liệu huấn luyện, giá trị đặc trưng được lấy đúng tại thời điểm trước khi sự kiện xảy ra, tránh rò rỉ dữ liệu từ tương lai.
ETL	(Extract, Transform, Load) Quy trình trích xuất dữ liệu từ nhiều nguồn, biến đổi (làm sạch, tổng hợp) và nạp vào một kho lưu trữ đích như Data Warehouse.
CI/CD	(Continuous Integration/Deployment) Quy trình tự động hóa việc tích hợp, kiểm thử (CI) và triển khai (CD) các thay đổi của hệ thống ra môi trường vận hành.
TTL	(Time To Live) Một tham số xác định khoảng thời gian một dữ liệu được coi là hợp lệ hoặc được lưu trữ trước khi bị xóa hoặc hết hạn.

Join Key	Một hoặc nhiều cột được sử dụng để kết hợp (join) dữ liệu từ các nguồn khác nhau. Trong Feast, Entity đóng vai trò là join key.
Latency	(Độ trễ) Thời gian cần thiết để hệ thống xử lý và trả về một kết quả sau khi nhận được yêu cầu. Trong MLOps, độ trễ thấp là yếu tố quan trọng cho các ứng dụng thời gian thực.
Throughput	(Thông lượng) Số lượng yêu cầu mà hệ thống có thể xử lý trong một đơn vị thời gian (ví dụ: số dự đoán mỗi giây).
Staleness	(Độ cũ/trễ của dữ liệu) Khoảng thời gian chênh lệch giữa thời điểm dữ liệu được tạo ra và thời điểm nó sẵn sàng để được sử dụng/phục vụ.

I.1. Từ Machine Learning đến MLOps

Trong môi trường học thuật hoặc thử nghiệm, quy trình Machine Learning thường nhằm tối ưu một mô hình trên một bộ dữ liệu tĩnh. Quy trình này thường diễn ra tuần tự qua các bước:

1. **Thu thập và xử lý dữ liệu:** Dữ liệu được thu thập và tiền xử lý một lần để chuẩn bị cho việc huấn luyện.
2. **Xử lý đặc trưng:** Các đặc trưng được tạo ra, lựa chọn và thử nghiệm thủ công.
3. **Huấn luyện và đánh giá mô hình:** Các nhà khoa học dữ liệu thử nghiệm nhiều thuật toán và tinh chỉnh siêu tham số để tìm ra mô hình tốt nhất.
4. **Triển khai:** Nếu có, bước này thường chỉ là một hoạt động riêng lẻ, ví dụ như xây dựng một API, giao diện đơn giản để demo.

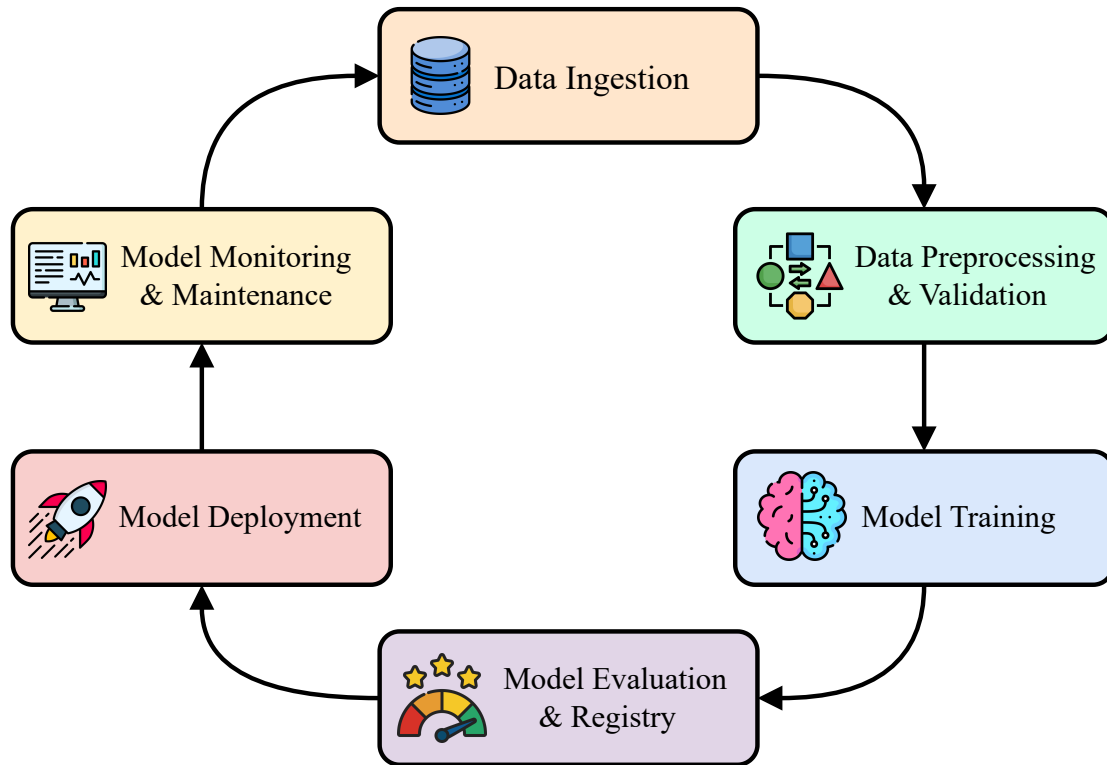
Quy trình trên phù hợp cho việc nghiên cứu nhưng không đủ mạnh mẽ và linh hoạt để vận hành trong thế giới thực, nơi dữ liệu liên tục thay đổi và hệ thống đòi hỏi độ tin cậy, khả năng lặp lại kết quả và khả năng quan sát. Để giải quyết những thách thức này và xây dựng các hệ thống Machine Learning bền vững, ngành công nghiệp đã áp dụng một triết lý vận hành gọi là MLOps.

MLOps (Machine Learning Operations) chuyển trọng tâm từ việc tạo ra một mô hình đơn lẻ sang việc xây dựng một hệ thống tự động và tuần hoàn có khả năng liên tục cung cấp và duy trì các sản phẩm Machine Learning. MLOps kế thừa các nguyên tắc cốt lõi từ DevOps và điều chỉnh chúng cho phù hợp với vòng đời đặc thù của Machine Learning:

- **CI (Continuous Integration):** Không chỉ tích hợp mã nguồn, CI trong MLOps còn bao gồm việc kiểm thử và xác thực tự động cho cả dữ liệu và các thành phần của mô hình.
- **CD (Continuous Delivery/Deployment):** Tự động hóa quá trình đóng gói và triển khai an toàn toàn bộ hệ thống (bao gồm mô hình, mã nguồn, và cấu hình) ra môi trường sản phẩm.
- **CT (Continuous Training):** Một khái niệm đặc thù của MLOps, chỉ việc tự động huấn luyện lại mô hình khi có các yếu tố kích hoạt, ví dụ như có dữ liệu mới, hiệu suất mô hình suy giảm, hoặc theo lịch.

Một vòng đời MLOps đầy đủ bao gồm các giai đoạn được tích hợp chặt chẽ với nhau:

- 📊 **Data Ingestion:** Đây là một pipeline dữ liệu liên tục, tự động thu thập dữ liệu mới từ các nguồn như streaming events, cơ sở dữ liệu, hay data lake.
- 🔧 **Data Preprocessing & Validation:** Dữ liệu mới được tự động làm sạch và biến đổi bằng các thư viện quen thuộc như **Pandas**. Quan trọng hơn, dữ liệu sẽ được xác thực để kiểm tra chất lượng, định dạng và tính hợp lệ, đảm bảo chỉ dữ liệu tốt mới được đưa vào huấn luyện.
- 🧠 **Model Training:** Quá trình huấn luyện được tự động hóa và kích hoạt theo lịch trình hoặc sự kiện. Các thông tin về thử nghiệm, tham số và chỉ số đánh giá được ghi nhận một cách có hệ thống bằng các công cụ theo dõi như **MLflow**.



Hình 2: Vòng đời MLOps tuần hoàn. Các giai đoạn được tự động hóa và kết nối với nhau, với trọng tâm là giám sát và tái huấn luyện liên tục để duy trì và cải thiện hiệu suất hệ thống.

- ✔ **Model Evaluation & Registry:** Mô hình mới được tự động đánh giá và so sánh với phiên bản đang chạy. Nếu tốt hơn, nó sẽ được đăng ký vào một *Model Registry* (ví dụ: cũng trong **MLflow**) để quản lý phiên bản và chuẩn bị cho việc triển khai.
- 🚀 **Model Deployment:** Mô hình được tự động đóng gói (ví dụ: **Docker**) và triển khai lên môi trường sản phẩm thông qua một quy trình CI/CD. Việc phục vụ mô hình ở quy mô lớn thường được quản lý bởi các nền tảng như **Kubernetes** hoặc các dịch vụ đám mây.
- 📊 **Model Monitoring & Maintenance:** Sau khi triển khai, mô hình được giám sát liên tục. Các hệ thống như **Prometheus** và **Grafana** được dùng để theo dõi các chỉ số vận hành và hiệu suất dự đoán. Khi phát hiện sự suy giảm chất lượng của mô hình, hệ thống sẽ gửi cảnh báo hoặc tự động kích hoạt lại vòng đời MLOps, bắt đầu bằng việc tái huấn luyện trên dữ liệu mới. Toàn bộ quy trình này được điều phối bởi các công cụ như **Airflow**.

Tóm lại, MLOps là một hệ sinh thái gồm nhiều thành phần quan trọng phối hợp nhịp nhàng. Trong đó, việc quản lý dữ liệu và đặc trưng một cách nhất quán cho cả môi trường huấn luyện và phục vụ là một thách thức nền tảng. Đây chính là lý do mà **Feature Store** ra đời và trở thành một trụ cột không thể thiếu trong các hệ thống MLOps hiện đại. Trong các phần tiếp theo, chúng ta sẽ tập trung đi sâu vào thành phần thiết yếu này.

I.2. Những thách thức trong quản lý đặc trưng

Một vòng đời MLOps tự động hóa và hiệu quả như đã trình bày ở trên phụ thuộc rất nhiều vào chất lượng và tính nhất quán của “nhiên liệu” đầu vào: các **đặc trưng**. Khi các quy trình CI/CD/CT được thiết lập, việc quản lý chính các đặc trưng trở thành một thách thức cốt lõi. Trong khi các công cụ như Airflow điều phối luồng công việc và MLflow theo dõi các thử nghiệm, một câu hỏi quan trọng vẫn còn bỏ ngỏ: “Thành phần nào trong hệ thống MLOps sẽ chịu trách nhiệm chuẩn hóa, lưu trữ, và cung cấp các đặc trưng một cách đáng tin cậy?”. Việc thiếu một thành phần chuyên biệt cho nhiệm vụ này sẽ dẫn đến các vấn đề nghiêm trọng cả về mặt kỹ thuật lẫn quy trình vận hành trong tổ chức.

🔗 Đảm bảo tính nhất quán logic giữa các môi trường

Đây là rào cản kỹ thuật phức tạp và phổ biến nhất, thường được biết đến với tên gọi **Training-Serving Skew**.

- **Bối cảnh:** Trong MLOps, pipeline training (thường xử lý theo lô, offline) và pipeline serving (xử lý theo thời gian thực, online) là hai hệ thống độc lập. Chúng có thể được viết bằng các ngôn ngữ khác nhau, chạy trên hạ tầng khác nhau và có yêu cầu về độ trễ hoàn toàn khác nhau.
- **Vấn đề:** Việc đảm bảo logic tính toán đặc trưng (ví dụ: cách xử lý giá trị null, làm tròn số, tổng hợp dữ liệu) giống hệt nhau 100% trên hai môi trường này là cực kỳ khó khăn và dễ phát sinh lỗi.
- **Hệ quả:** Chỉ một khác biệt nhỏ trong logic cũng đủ để làm hiệu suất của mô hình trên thực tế suy giảm nghiêm trọng so với kết quả đánh giá và gây ảnh hưởng trực tiếp đến giá trị kinh doanh.

👥 Tối ưu hóa việc phát triển và cộng tác

Khi một tổ chức mở rộng quy mô, việc thiếu một nền tảng quản lý đặc trưng chung sẽ tạo ra các rào cản lớn về hiệu suất và sự hợp tác giữa các nhóm.

- **Vấn đề:** Mỗi nhóm/dự án có xu hướng tự xây dựng lại các đặc trưng từ đầu. Các định nghĩa đặc trưng bị phân tán trong vô số mã nguồn, notebook và tài liệu, biến kiến thức này thành tài sản riêng của một vài cá nhân.
- **Hệ quả:**
 - **Lãng phí tài nguyên:** Các nhóm tốn thời gian và công sức để thực hiện các công việc tính toán trùng lặp.
 - **Thiếu khả năng tái sử dụng và khám phá:** Rất khó để một thành viên mới hoặc một nhóm khác biết được đặc trưng nào đã tồn tại để có thể tận dụng, làm chậm đáng kể tốc độ phát triển mô hình mới.
 - **Định nghĩa không nhất quán:** Cùng một khái niệm kinh doanh (ví dụ: “khách hàng thân thiết”) có thể có nhiều định nghĩa và cách tính toán khác nhau trong các dự án, dẫn đến sự thiếu nhất quán và khó so sánh kết quả.

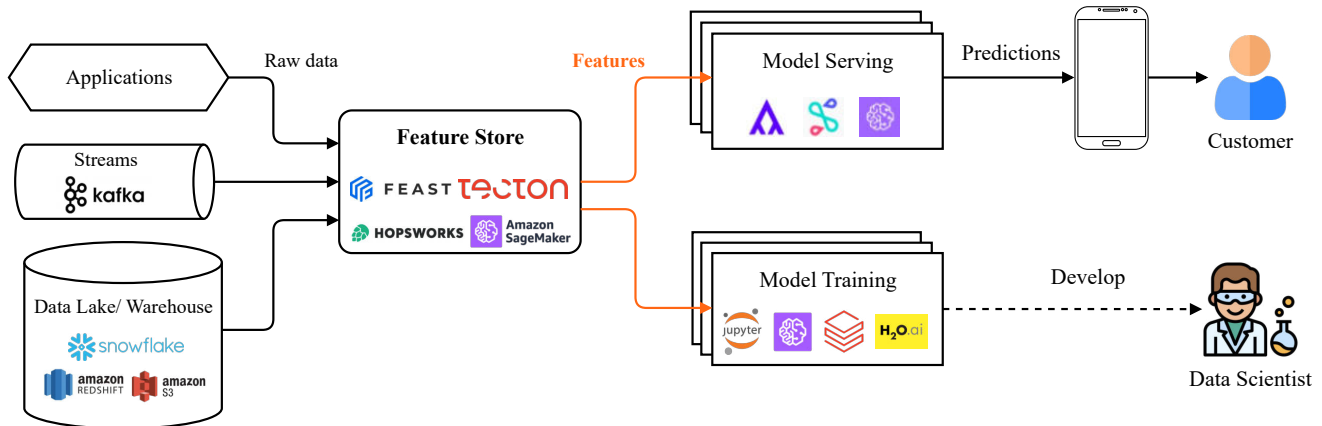
Những thách thức trên cho thấy rõ, một hệ thống MLOps hoàn chỉnh không chỉ cần các pipeline tự động mà còn đòi hỏi một lớp hạ tầng chuyên biệt cho dữ liệu đặc trưng. Lớp hạ tầng này đảm bảo tính nhất quán, khả năng tái sử dụng và quản trị hiệu quả các đặc trưng. Đây chính là vai trò mà **Feature Store** đảm nhiệm trong kiến trúc MLOps hiện đại.

II. Tổng quan về Feature Store

Để giải quyết các thách thức về kỹ thuật và tổ chức đã nêu, **Feature Store** được tích hợp vào kiến trúc MLOps như một lớp hạ tầng dữ liệu chuyên biệt cho Machine Learning. Nó không phải là một công cụ độc lập, mà là cầu nối trung tâm giữa dữ liệu và mô hình, giúp các tổ chức giảm thiểu sự trùng lặp trong công việc xử lý dữ liệu, tăng tốc vòng đời phát triển Machine Learning và thúc đẩy khả năng cộng tác giữa các nhóm khoa học dữ liệu.

II.1. Vai trò và vị trí trong kiến trúc MLOps

Về bản chất, Feature Store hoạt động như một nền tảng trung tâm cho dữ liệu và siêu dữ liệu của đặc trưng trong suốt vòng đời của một dự án. Nó phục vụ cho hai nhu cầu hoàn toàn khác biệt: xử lý theo lô (batch) cho việc huấn luyện và xử lý thời gian thực (real-time) cho việc dự đoán. Sơ đồ dưới đây minh họa vị trí và luồng dữ liệu của nó trong một kiến trúc tổng thể.



Hình 3: Kiến trúc tổng quan của Feature Store trong MLOps. Nó đóng vai trò trung tâm, kết nối các nguồn dữ liệu, các pipeline xử lý đặc trưng với các ứng dụng Machine Learning.

Như trong Hình 3, luồng dữ liệu và hoạt động của Feature Store có thể được tóm tắt như sau:

- **Đầu vào:** Các pipeline kỹ thuật đặc trưng đọc dữ liệu thô từ nhiều nguồn (Data Lake, Data Warehouse, Streaming Platforms) để tạo ra các giá trị đặc trưng.
- **Nạp dữ liệu:** Các giá trị đặc trưng được nạp vào Feature Store và lưu trữ đồng thời ở hai nơi:
 - Toàn bộ lịch sử giá trị được ghi vào **Offline Store** để huấn luyện mô hình.
 - Giá trị mới nhất được ghi vào **Online Store** để dự đoán thời gian thực.

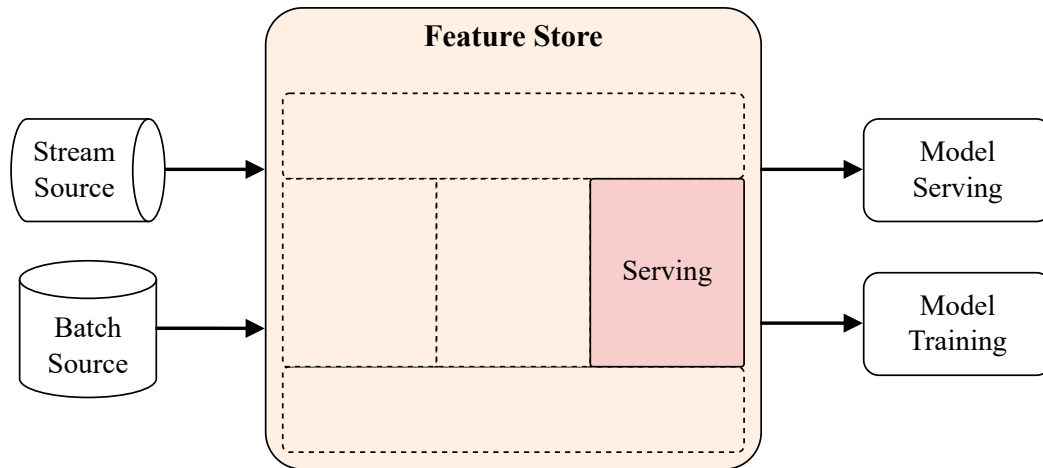
- **Đầu ra:**

- Các **quy trình huấn luyện** truy vấn Offline Store để lấy các bộ dữ liệu lịch sử lớn.
- Các **dịch vụ dự đoán** truy vấn Online Store để lấy vector đặc trưng mới nhất với độ trễ thấp.

II.2. Các thành phần cốt lõi

Một Feature Store hiện đại thường bao gồm 5 thành phần chính, hoạt động như một giao diện giữa dữ liệu và mô hình. Mỗi thành phần đều đóng vai trò quan trọng trong việc giải quyết các thách thức của MLOps, từ đó cho phép các nhóm cộng tác hiệu quả và tăng tốc vòng đời phát triển Machine Learning.

🚀 Serving

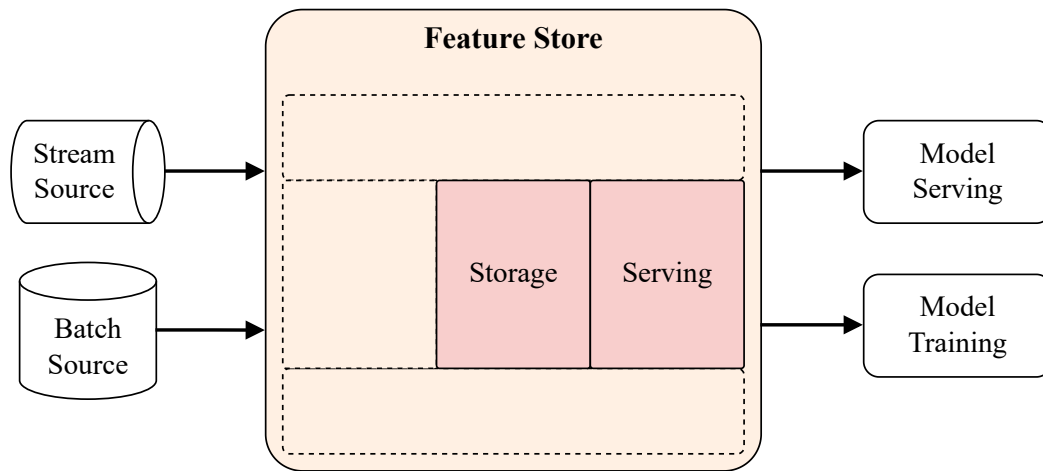


Hình 4: Lớp Serving cung cấp dữ liệu đặc trưng nhất quán cho cả môi trường huấn luyện (offline) và dự đoán (online).

Đây là chức năng cung cấp dữ liệu đặc trưng cho các mô hình, với yêu cầu cốt lõi là phải đảm bảo một “view” nhất quán giữa các môi trường. Định nghĩa của đặc trưng được dùng để huấn luyện mô hình phải khớp chính xác 100% với đặc trưng được cung cấp khi dự đoán. Khi chúng không khớp, hiện tượng **training-serving skew** sẽ xảy ra, gây sụt giảm hiệu năng nghiêm trọng và rất khó để gỡ lỗi.

- **Offline Serving:** Cung cấp SDK (thường bằng Python) để truy vấn các tập dữ liệu lịch sử lớn cho việc huấn luyện. Chức năng này hỗ trợ các phép join point-in-time correct, cho phép “du hành thời gian” để lấy đúng giá trị của đặc trưng tại một thời điểm trong quá khứ, đảm bảo không bị rò rỉ dữ liệu.
- **Online Serving:** Cung cấp một API hiệu năng cao để lấy một vector đặc trưng (gồm các giá trị mới nhất) với độ trễ thấp. Các yêu cầu này được phục vụ bởi một cơ sở dữ liệu tốc độ cao.

Storage

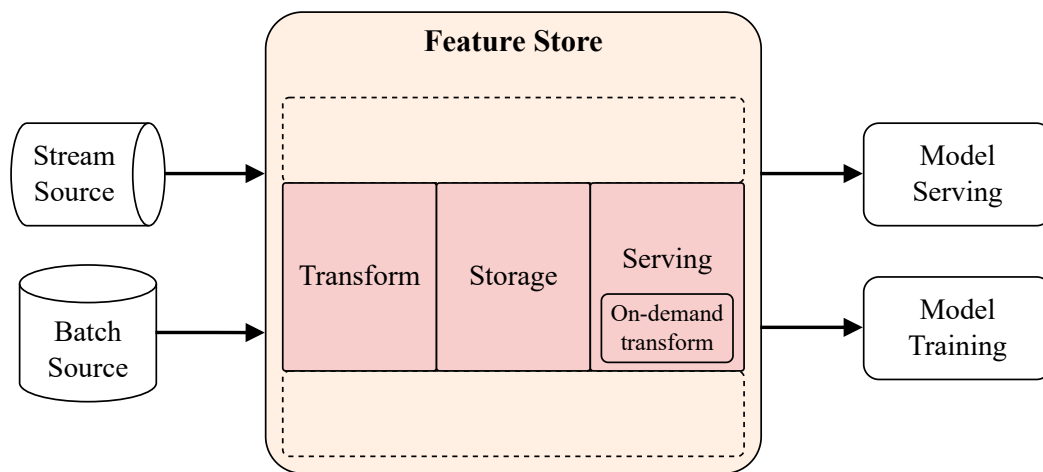


Hình 5: Lớp Storage bao gồm Offline Store cho dữ liệu lịch sử và Online Store cho dữ liệu mới nhất.

Feature Store trừu tượng hóa lớp lưu trữ, cung cấp hai kho chứa được tối ưu cho các mục đích sử dụng khác nhau:

- **Offline Store:** Lưu trữ dữ liệu lịch sử của đặc trưng trong nhiều tháng hoặc nhiều năm để huấn luyện mô hình. Kho này thường được xây dựng trên các Data Warehouse (BigQuery, Snowflake) hoặc Data Lake (S3) có sẵn của tổ chức.
- **Online Store:** Chỉ lưu trữ giá trị mới nhất của các đặc trưng cho mỗi thực thể để phục vụ truy vấn với độ trễ thấp khi dự đoán. Kho này thường được triển khai bằng các cơ sở dữ liệu Key-Value hiệu năng cao như Redis hoặc DynamoDB.

Transformation

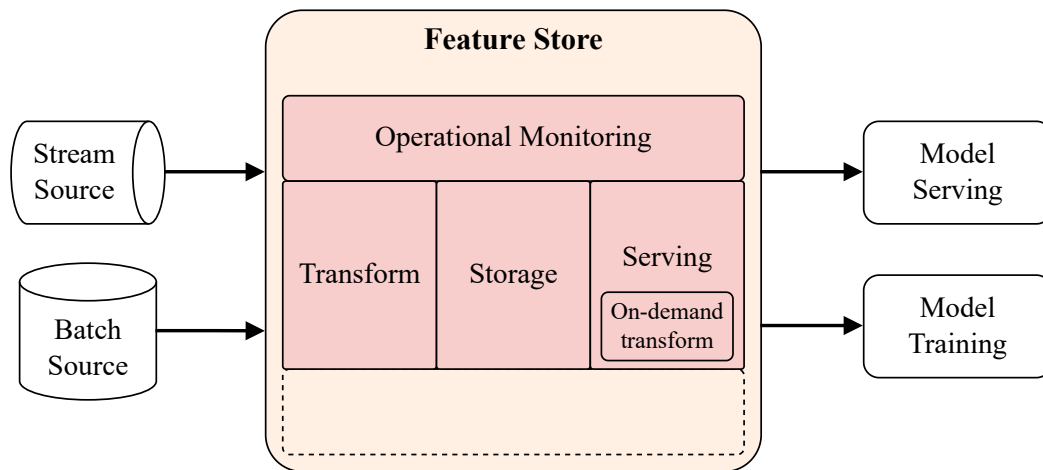


Hình 6: Lớp Transformation quản lý và điều phối các pipeline tính toán đặc trưng từ dữ liệu thô.

Feature Store quản lý và điều phối các tác vụ biến đổi dữ liệu để tính toán ra các giá trị đặc trưng. Việc tái sử dụng mã nguồn biến đổi giữa các môi trường là chìa khóa để giải quyết triệt để vấn đề **Training-Serving Skew**. Có ba loại hình biến đổi chính:

- **Batch Transformations:** Áp dụng trên dữ liệu tĩnh (dữ liệu lịch sử) từ Data Warehouse hoặc Data Lake.
- **Streaming Transformations:** Áp dụng trên các nguồn dữ liệu dòng chảy (như Kafka) để tính toán đặc trưng gần thời gian thực.
- **On-demand Transformations:** Tính toán đặc trưng “theo yêu cầu” tại thời điểm dự đoán, sử dụng những dữ liệu chỉ có sẵn tại lúc yêu cầu (ví dụ: vị trí hiện tại của người dùng).

📊 Monitoring

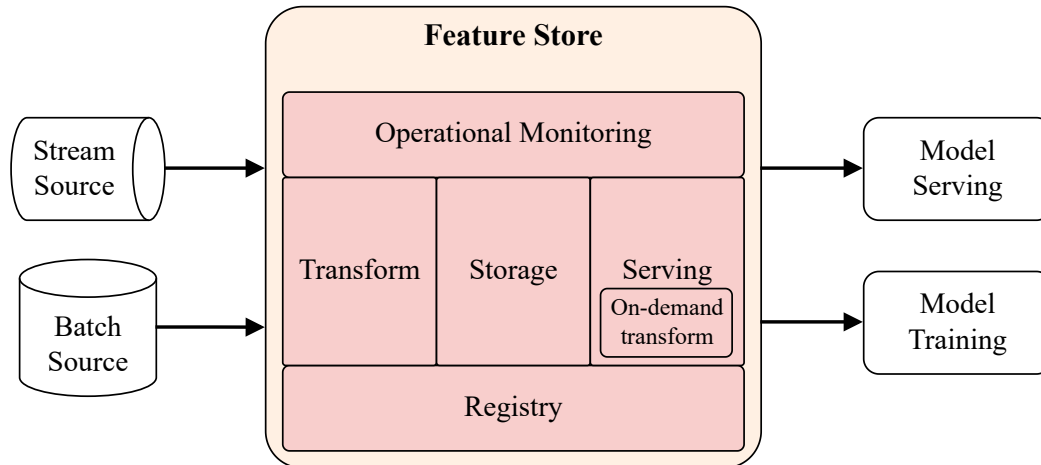


Hình 7: Lớp Monitoring theo dõi chất lượng dữ liệu và sức khỏe vận hành của hệ thống.

Khi xảy ra sự cố trong hệ thống Machine Learning, nguyên nhân thường đến từ dữ liệu. Feature Store có vị trí đặc biệt thuận lợi để phát hiện và cảnh báo những vấn đề này.

- **Giám sát chất lượng dữ liệu:** Theo dõi data drift và so sánh dữ liệu phục vụ với dữ liệu huấn luyện để phát hiện training-serving skew.
- **Giám sát vận hành:** Theo dõi các chỉ số hệ thống như latency, throughput, tỷ lệ lỗi của dịch vụ phục vụ, hay độ mới của dữ liệu.

☰ Registry



Hình 8: Registry là trung tâm điều phối, đóng vai trò là nguồn duy nhất cho các thành phần khác.

Registry là thành phần trung tâm, đóng vai trò là nguồn duy nhất cho mọi thông tin về đặc trưng trong hệ thống.

- **Danh mục chung:** Các nhóm sử dụng Registry như một catalog để khám phá, cộng tác và chia sẻ các định nghĩa đặc trưng, tránh lãng phí công sức phát triển lại các đặc trưng đã có.
- **Cấu hình hệ thống:** Các định nghĩa trong Registry được các thành phần khác (Transformation, Storage, Serving) sử dụng để tự động hóa các tác vụ. Nó cũng lưu trữ thông tin về phiên bản và nguồn gốc để phục vụ việc gỡ lỗi và kiểm toán.

II.3. Các Feature Store phổ biến trên thị trường

Hệ sinh thái Feature Store đang phát triển nhanh chóng với nhiều giải pháp khác nhau:

II.3.1. Giải pháp thương mại và dịch vụ đám mây

- **Tecton:** Được xem là giải pháp enterprise hàng đầu, phát triển bởi đội ngũ đã tạo ra Michelangelo Feature Store tại Uber. Rất mạnh mẽ, hoàn chỉnh nhưng là một sản phẩm thương mại.
- **Vertex AI Feature Store (Google Cloud) & SageMaker Feature Store (AWS):** Là các dịch vụ được quản lý, tích hợp sâu vào hệ sinh thái cloud của nhà cung cấp. Đây là lựa chọn tốt nếu tổ chức đã cam kết sử dụng một nền tảng cloud cụ thể.

II.3.2. Giải pháp mã nguồn mở

- **Feast (Feature Store for Machine Learning):** Đã nhanh chóng trở thành một tiêu chuẩn mã nguồn mở hàng đầu. Điểm mạnh lớn nhất của Feast là tính linh hoạt và không bị ràng buộc vào nhà cung cấp. Nó không phải là một nền tảng “tất cả trong một” mà là một lớp điều phối mỏng, cho phép bạn tận dụng chính hạ tầng dữ liệu sẵn có của mình. Ví dụ như Snowflake, BigQuery cho offline store; Redis, DynamoDB cho online store.

II.4. Feast

Sau khi đã hiểu các thành phần lý thuyết của một Feature Store, chúng ta sẽ tìm hiểu về **Feast** và triết lý thiết kế của công cụ này giúp định hình cách chúng ta sử dụng nó trong thực tế.

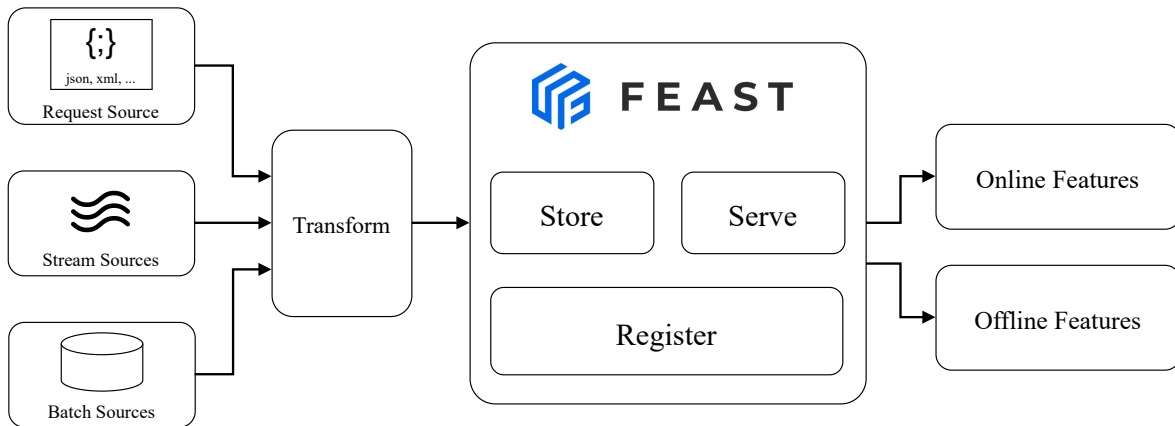
II.4.1. Triết lý cốt lõi

Điểm khác biệt lớn nhất và cũng là điểm mạnh nhất của Feast là nó không phải là một nền tảng “tất cả trong một”. Thay vì xây dựng một hệ thống công kênh với pipeline, cơ sở dữ liệu, và công cụ điều phối riêng, Feast được thiết kế như một lớp điều phối mỏng linh hoạt. Triết lý của Feast là kết nối và tái sử dụng chính hạ tầng dữ liệu mà tổ chức đã có sẵn.

- Nó không thay thế Data Warehouse (như BigQuery, Snowflake) mà sử dụng chúng làm **Offline Store**.
- Nó không thay thế cơ sở dữ liệu Key-Value (như Redis, DynamoDB) mà sử dụng chúng làm **Online Store**.
- Nó không thay thế công cụ điều phối (như Airflow) mà tích hợp với chúng để lên lịch cho các tác vụ nạp dữ liệu.

Cách tiếp cận này giúp các đội MLOps có thể nhanh chóng triển khai một Feature Store mà không cần phải xây dựng lại toàn bộ hệ thống dữ liệu từ đầu.

II.4.2. Vai trò và phạm vi



Hình 9: Feast tiếp nhận dữ liệu đã được biến đổi, sau đó quản lý, lưu trữ và phục vụ chúng thành các đặc trưng online (cho dự đoán) và offline (cho huấn luyện).

Như được minh họa trong Hình 9, vai trò của Feast là một lớp giao diện chuyên biệt cho dữ liệu Machine Learning.

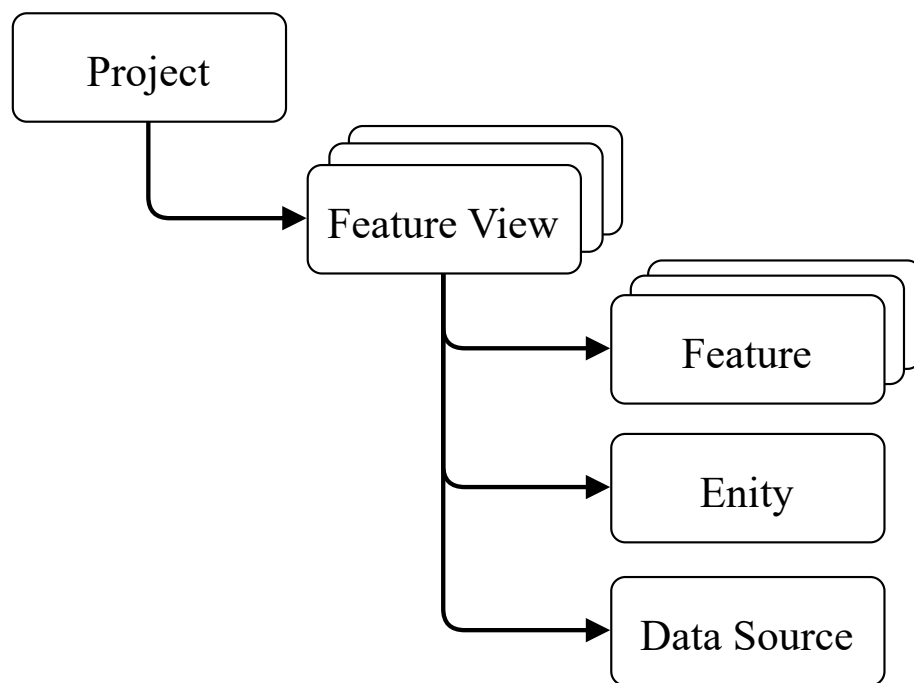
- **Feast** là một lớp truy cập dữ liệu thống nhất, giúp đảm bảo tính nhất quán giữa training và serving (với các phép join point-in-time correct) và hoạt động như một trung tâm registry để thúc đẩy việc cộng tác và tái sử dụng đặc trưng.

- **Feast không phải là** một hệ thống dữ liệu đa năng. Nó không thay thế các công cụ ETL (như dbt, Spark), công cụ điều phối (như Airflow), hay các hệ thống lưu trữ (như Data Warehouse, Database) mà được thiết kế để tích hợp và hoạt động bên trên chúng.

II.4.3. Cú pháp và cách sử dụng cơ bản

Trước khi đi vào ví dụ chi tiết, chúng ta cần làm quen với các khái niệm và lệnh cốt lõi khi làm việc với Feast. Tương tác với Feast chủ yếu diễn ra qua hai hình thức: khai báo các đối tượng bằng Python và sử dụng CLI/SDK để quản lý và truy vấn dữ liệu.

Các đối tượng định nghĩa chính Đây là các khối xây dựng cơ bản mà bạn sẽ khai báo bằng Python trong một “Feature Repository”. Hình 10 minh họa cấu trúc phân cấp của các đối tượng này.



Hình 10: Cấu trúc phân cấp của một project trong Feast. Mỗi project chứa các Feature View, và mỗi Feature View liên kết các Feature với một Entity và một Data Source.

- **Project:** Là không gian tên (namespace) ở cấp cao nhất. Mỗi project là một vũ trụ riêng biệt chứa các thực thể và đặc trưng, giúp cô lập hoàn toàn các môi trường khác nhau (ví dụ: dev, staging, production).
- **DataSource:** Đối tượng chỉ định nguồn dữ liệu vật lý nơi các giá trị đặc trưng được lưu trữ. Đây có thể là một file Parquet, một bảng trong BigQuery, hay một topic Kafka.
- **Entity:** Đại diện cho đối tượng kinh doanh mà các đặc trưng mô tả (ví dụ: `customer_id`, `product_id`). Đây chính là khóa (join key) để liên kết và tra cứu đặc trưng.
- **FeatureView:** Là thành phần trung tâm, định nghĩa một nhóm logic các đặc trưng. Một FeatureView sẽ:

- Chứa định nghĩa của một hoặc nhiều **Features** (tên, kiểu dữ liệu).
 - Liên kết các features này với một hoặc nhiều **Entity**.
 - Trỏ đến một **DataSource** để Feast biết nơi lấy dữ liệu.
- **FeatureService**: Dùng để nhóm các đặc trưng từ một hoặc nhiều **FeatureView** lại với nhau, phục vụ cho một phiên bản model hoặc một ứng dụng cụ thể. Đây là một cách thực hành tốt để quản lý vòng đời và các phụ thuộc dữ liệu của model.

Các lệnh CLI phổ biến Đây là các lệnh bạn dùng trên terminal để quản lý vòng đời của Feature Store:

- **feast init <tên_project>**: Khởi tạo một cấu trúc thư mục project Feast mẫu (một Feature Repository).
- **feast apply**: Đọc các định nghĩa Python trong repo, đăng ký chúng vào registry, và cập nhật hạ tầng (ví dụ: tạo bảng trong Online Store).
- **feast materialize <ngày_bắt_đầu> <ngày_kết_thúc>**: Nạp dữ liệu đặc trưng từ Offline Store vào Online Store trong một khoảng thời gian nhất định để chuẩn bị cho việc phục vụ online.
- **feast ui**: Khởi chạy một giao diện web để khám phá các đối tượng đã được đăng ký trong Feature Store.

Các hàm chính để truy vấn dữ liệu Đây là các hàm Python bạn dùng trong code để lấy dữ liệu cho các tác vụ Machine Learning:

- **store.get_historical_features(...)**: Dùng để lấy dữ liệu lịch sử từ **Offline Store** nhằm tạo ra bộ dữ liệu huấn luyện hoặc thực hiện dự đoán theo lô. Hàm này thực hiện các phép join point-in-time correct để chống rò rỉ dữ liệu.
- **store.get_online_features(...)**: Dùng để lấy vector đặc trưng mới nhất từ **Online Store** với độ trễ thấp, phục vụ cho việc dự đoán thời gian thực.

Với những khái niệm cơ bản này, chúng ta đã sẵn sàng để áp dụng chúng vào bài toán thực tế để xây dựng một hệ thống phát hiện gian lận.

III. Thực hành

Mô tả bài toán

Gian lận thẻ tín dụng là một vấn đề nghiêm trọng, gây thiệt hại tài chính lớn cho cả khách hàng và các tổ chức tài chính. Do đó, việc xây dựng các hệ thống có khả năng phát hiện giao dịch đáng ngờ một cách nhanh chóng và chính xác là cực kỳ quan trọng. Mục tiêu của chúng ta là xây dựng một mô hình Machine Learning có khả năng phân loại giao dịch là hợp lệ hay gian lận gần như ngay lập tức.

Dữ liệu và mục tiêu

Chúng ta sẽ sử dụng bộ dữ liệu **Credit Card Fraud Detection** trên Kaggle¹. Dữ liệu này chứa các giao dịch thẻ tín dụng ẩn danh được thực hiện trong hai ngày tại Châu Âu. Đây là một bài toán phân loại nhị phân, với cấu trúc dữ liệu như sau:

Tên cột	Kiểu dữ liệu	Mô tả
<i>Đặc trưng đầu vào (Input Features)</i>		
Time	Số (Numeric)	Thời gian tương đối (giây) giữa giao dịch hiện tại và giao dịch đầu tiên.
V1-V28	Số (Numeric)	Các đặc trưng ẩn danh (kết quả của biến đổi PCA) để bảo vệ quyền riêng tư.
Amount	Số (Numeric)	Số tiền của giao dịch.
<i>Biến mục tiêu (Output/Target)</i>		
Class	Nhị phân (0/1)	1 cho giao dịch gian lận, 0 cho giao dịch hợp lệ.

Bảng 2: Mô tả các trường trong bộ dữ liệu Credit Card Fraud Detection.

Ngoài ra, bộ dữ liệu này còn có những thách thức thực tế mà chúng ta cần lưu ý:

- **Mất cân bằng nghiêm trọng:** Đúng với bản chất của bài toán gian lận, tỷ lệ giao dịch lừa đảo (**Class=1**) là cực kỳ nhỏ so với tổng số giao dịch, đặt ra một thách thức cho việc huấn luyện và đánh giá mô hình.
- **Thiếu dấu thời gian thực (Timestamp):** Cột **Time** chỉ là số giây chênh lệch, không phải là một dấu thời gian cụ thể (ví dụ: 2023-10-27 10:00:00). Đây là một vấn đề phổ biến và chúng ta sẽ phải xử lý nó, vì các hệ thống Feature Store yêu cầu timestamp để đảm bảo tính chính xác của dữ liệu theo thời gian.

Mục tiêu thực hành

Mục tiêu của phần thực hành này không chỉ là huấn luyện một mô hình, mà là xây dựng một quy trình MLOps hoàn chỉnh ở quy mô nhỏ bằng **Feast**. Cụ thể, chúng ta sẽ mô phỏng vòng đời của một đặc trưng trong môi trường thực tế:

¹<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

1. **Định nghĩa và Đăng ký:** Khai báo các đặc trưng (V1-V28, Amount) trong một *Feature Repository* tập trung.
2. **Phục vụ Offline:** Sử dụng Feast để tạo ra một bộ dữ liệu huấn luyện point-in-time correct), giải quyết rủi ro rò rỉ dữ liệu.
3. **Phục vụ Online:** Sau khi có mô hình, chúng ta sẽ mô phỏng việc một ứng dụng real-time truy vấn các đặc trưng mới nhất từ *Online Store* để đưa ra dự đoán.

Bước 0: Khởi tạo dự án và chuẩn bị Dữ liệu

Cài đặt các thư viện cần thiết

Chúng ta cần cài đặt các thư viện phụ trợ cho Feast và Machine Learning.

```
> pip install feast scikit-learn pandas pyarrow kagglehub
```

Khởi tạo dự án Feast

Thay vì tạo thư mục thủ công, chúng ta sẽ dùng ngay lệnh của Feast để khởi tạo toàn bộ cấu trúc dự án. Lệnh này sẽ tạo một thư mục dự án `feast-fraud-detection` chứa một feature repository mẫu bên trong.

```
> feast init feast-fraud-detection
```

Sau đó, di chuyển vào thư mục vừa tạo để bắt đầu làm việc.

```
> cd feast-fraud-detection
```

Tải dữ liệu

Để tự động hóa việc tải dữ liệu từ Kaggle, chúng ta tạo một script `download_data.py`. Script này sử dụng thư viện `kagglehub` để tải và di chuyển file CSV vào thư mục `data`.

download_data.py

```
1 import os
2 import shutil
3 import kagglehub
4 os.makedirs("data", exist_ok=True)
5 os.environ["KAGGLEHUB_CACHE"] = "data"
6
7 path = kagglehub.dataset_download("mlg-ulb/creditcardfraud")
8
9 csv_path = None
10 for root, dirs, files in os.walk(path):
11     for file in files:
12         if file.endswith(".csv"):
```

```

13         src = os.path.join(root, file)
14         dst = os.path.join("data", file)
15         shutil.move(src, dst)
16         csv_path = dst
17
18     for item in os.listdir("data"):
19         if item != os.path.basename(csv_path):
20             item_path = os.path.join("data", item)
21             if os.path.isdir(item_path):
22                 shutil.rmtree(item_path)
23             else:
24                 os.remove(item_path)
25     print(f"Dữ liệu đã được lưu tại: {csv_path}")

```

Chạy script để tải dữ liệu:

```
> python download_data.py
```

Tiền xử lý dữ liệu

Tiếp theo, chúng ta sẽ tạo và chạy script `prepare_data.py`. Script này thực hiện chuẩn bị dữ liệu thô cho Feast:

- **Tạo dấu thời gian:** Vì dữ liệu gốc thiếu thông tin thời gian thực, script sẽ tạo ra một cột `event_timestamp` giả lập. Đây là yêu cầu bắt buộc của Feast để đảm bảo tính chính xác tại từng thời điểm.
- **Tạo định danh duy nhất:** Script cũng tạo ra cột `transaction_id` để làm khóa định danh cho mỗi giao dịch.
- **Chuẩn hóa và lưu trữ:** Cuối cùng, nó đổi tên cột `target` thành `is_fraud` cho dễ hiểu và lưu toàn bộ dữ liệu sạch dưới định dạng Parquet để tối ưu hiệu suất.

prepare_data.py

```

1 import pandas as pd
2 from datetime import datetime, timedelta
3
4 df = pd.read_csv('data/creditcard.csv')
5
6 print("Tạo dữ liệu timestamp giả lập...")
7 now = datetime.now()
8 df['event_timestamp'] = df['Time'].apply(lambda x: now - timedelta(seconds=x))
9 df['transaction_id'] = df.index.astype(str)
10
11 print("Định dạng lại các cột...")
12 df.rename(columns={'Class': 'is_fraud'}, inplace=True)
13 final_df = df[[
14     'event_timestamp', 'transaction_id',
15     'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V10',
16     'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18', 'V19', 'V20',

```

```

17     'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27', 'V28',
18     'Amount', 'is_fraud'
19 ]]
20
21 print(f"Lưu trữ dữ liệu đã xử lý vào 'data/processed_data.parquet'...")
22 final_df.to_parquet('data/processed_data.parquet', index=False)

```

Chạy script:

```
> python prepare_data.py
```

Bước 1: Định nghĩa các thành phần trong Feature Repo

Đây là bước cốt lõi, nơi chúng ta sẽ giúp Feast hiểu về dữ liệu của mình bằng cách định nghĩa các thành phần cơ bản trong thư mục `feature_repo`.

Cấu trúc thư mục dự án

Sau khi khởi tạo và chuẩn bị dữ liệu, cấu trúc thư mục dự án sẽ tương tự như sau. Thư mục `feature_repo` chính là trung tâm nơi chúng ta sẽ làm việc ở bước này.

```

feast-fraud-detection/
├── data/
│   ├── creditcard.csv
│   └── processed_data.parquet
├── feature_repo/
│   ├── __init__.py
│   ├── data/
│   │   ├── registry.db
│   │   ├── feature_store.yaml
│   │   └── fraud_features.py
│   ├── download_data.py
│   ├── prepare_data.py
│   ├── train_model.py
│   └── predict_online.py

```

Chỉnh sửa `feature_store.yaml`

Lệnh `feast init` tạo ra một file cấu hình trung tâm là `feature_repo/feature_store.yaml`. File này định nghĩa kiến trúc cho Feature Store. Mở file và thay thế toàn bộ nội dung bằng đoạn sau:

feature_repo/feature_store.yaml

```

1 project: fraud_detection_demo
2 registry: data/registry.db
3 provider: local
4 online_store:
5     type: sqlite
6     path: data/online_store.db

```

- **project:** Tên định danh cho dự án.
- **registry:** Đường dẫn tới file CSDL (mặc định là SQLite) để lưu trữ tất cả các định nghĩa về feature đã được đăng ký.
- **provider:** Khai báo môi trường hoạt động. `local` có nghĩa là Feast sẽ sử dụng các tài nguyên trên máy cục bộ.
- **online_store:** Cấu hình cho kho dữ liệu online, nơi lưu trữ các feature mới nhất để phục vụ dự đoán thời gian thực. Ở đây chúng ta dùng SQLite để đơn giản hóa.

Định nghĩa đặc trưng trong `fraud_features.py`

Chúng ta sẽ định nghĩa các đặc trưng trong file `feature_repo/fraud_features.py`. File này là trái tim của Feature Store, nơi khai báo ba khái niệm quan trọng:

- **FileSource:** Chỉ cho Feast biết nguồn dữ liệu offline (dữ liệu lịch sử) nằm ở đâu. Trong trường hợp này là file `processed_data.parquet` chúng ta đã tạo.
- **Entity:** Đại diện cho một đối tượng cốt lõi mà các đặc trưng mô tả. Ở đây, mỗi `transaction_id` là một thực thể. Đây chính là khóa để Feast tra cứu và join dữ liệu.
- **FeatureView:** Là một nhóm các đặc trưng có liên quan logic. Nó liên kết một `Entity` với các đặc trưng của nó (`V1-V28, Amount`) và chỉ định nguồn dữ liệu (`FileSource`).

`feature_repo/fraud_features.py`

```

1 from datetime import timedelta
2 from feast import Entity, FeatureView, Field, FileSource, ValueType
3 from feast.types import Float32
4
5 # 1. Định nghĩa nguồn dữ liệu offline
6 fraud_data_source = FileSource(
7     path="../data/processed_data.parquet",
8     timestamp_field="event_timestamp",
9 )
10
11 # 2. Định nghĩa thực thể (Entity)
12 transaction = Entity(
13     name="transaction_id",
14     value_type=ValueType.STRING,
15     description="ID of a credit card transaction",
16 )
17
18 # 3. Định nghĩa Feature View để nhóm các đặc trưng
19 transaction_features_v1 = FeatureView(
20     name="transaction_features_v1",
21     entities=[transaction],
22     ttl=timedelta(days=365),
23     schema=[
24         Field(name=f"V{i}", dtype=Float32) for i in range(1, 29)
25     ] + [
26         Field(name="Amount", dtype=Float32),

```

```
27     ],
28     source=fraud_data_source,
29     online=True,
30     description="Anonymized transaction features and transaction amount",
31     tags={"owner": "fraud_detection_team", "version": "1"},
32 )
```

Bước 2: Apply và nạp dữ liệu vào Feature Store

Sau khi khai báo, chúng ta cần triển khai các định nghĩa này và nạp dữ liệu vào store. Quá trình này biến các khai báo trong code thành các đối tượng hoạt động được mà Feast có thể quản lý và phục vụ. **Lưu ý:** Các lệnh feast sau đây phải được chạy từ bên trong thư mục `feature_repo`.

```
> cd feature_repo
```

Apply các định nghĩa

Lệnh `feast apply` có vai trò như `git commit` cho các đặc trưng. Nó quét các file Python trong repo, đọc các định nghĩa (Entity, FeatureView...) và đăng ký chúng vào một metadata registry là file `registry.db`. Sau bước này, Feast đã “nhận biết” được các đặc trưng của bạn.

```
> feast apply
```

Materialize: Nạp dữ liệu vào online Store

Lệnh `feast materialize` thực hiện việc di chuyển dữ liệu. Nó sẽ đọc dữ liệu lịch sử từ Offline Store (file Parquet), xác định giá trị *mới nhất* của từng đặc trưng cho mỗi thực thể, và nạp chúng vào Online Store (file SQLite). Quá trình này giúp Online Store luôn có sẵn dữ liệu mới nhất để phục vụ các yêu cầu dự đoán thời gian thực với độ trễ thấp.

```
> feast materialize $(date -d '1 year ago' +%Y-%m-%d) $(date +%Y-%m-%d)
```

Lưu ý cho người dùng Windows: Lệnh `date` ở trên sẽ không hoạt động. Bạn cần thay thế ngày bắt đầu và kết thúc theo định dạng YYYY-MM-DD một cách thủ công. Ví dụ:

```
> feast materialize 2024-10-05 2025-10-05
```

Sau khi hoàn tất, hãy quay trở lại thư mục gốc của dự án.

```
> cd ..
```

Bước 3: Lấy dữ liệu offline và huấn luyện mô hình

Đây là lúc Feast thể hiện sức mạnh trong việc tạo ra bộ dữ liệu huấn luyện một cách an toàn và đáng tin cậy. Script `train_model.py` sẽ mô phỏng quy trình của một nhà khoa học dữ liệu khi cần huấn luyện mô hình.

Điểm cốt lõi ở đây là hàm `get_historical_features`. Thay vì join dữ liệu thủ công, chúng ta yêu cầu Feast thực hiện. Feast sẽ thực hiện một phép join point-in-time correct, đảm bảo rằng tại mỗi dòng dữ liệu sự kiện, các giá trị đặc trưng được lấy đúng tại thời điểm *trước khi* sự kiện đó xảy ra. Điều này giúp ngăn chặn rò rỉ dữ liệu từ tương lai, một lỗi phổ biến và nghiêm trọng khi xây dựng mô hình.

train_model.py

```

1 import pandas as pd
2 from feast import FeatureStore
3 from sklearn.model_selection import train_test_split
4 from sklearn.linear_model import LogisticRegression
5 from sklearn.metrics import classification_report
6 import joblib
7
8 # 1. LẤY DỮ LIỆU TRAINING TỪ OFFLINE STORE CỦA FEAST
9 store = FeatureStore(repo_path="feature_repo")
10
11 # entity_df chứa danh sách các thực thể và timestamp của sự kiện
12 entity_df_source = pd.read_parquet("data/processed_data.parquet")
13 entity_df = entity_df_source[['event_timestamp', 'transaction_id', 'is_fraud']]
14
15 # Định nghĩa các feature cần lấy
16 feature_names = [f"transaction_features_v1:V{i}" for i in range(1, 29)]
17 feature_names.append("transaction_features_v1:Amount")
18
19 print("Lấy dữ liệu training từ Offline Store...")
20 training_data = store.get_historical_features(
21     entity_df=entity_df,
22     features=feature_names,
23 ).to_df()
24
25 # 2. HUẤN LUYỆN MÔ HÌNH
26 print("\nBắt đầu huấn luyện mô hình...")
27 y = training_data['is_fraud']
28 X = training_data.drop(columns=['event_timestamp', 'transaction_id', 'is_fraud'], axis=
29                               1)
30
31 X_train, X_test, y_train, y_test = train_test_split(
32     X, y, test_size=0.2, random_state=42, stratify=y
33 )
34
35 model = LogisticRegression(max_iter=1000).fit(X_train, y_train)
36
37 print("\n--- Kết quả đánh giá mô hình ---")
38 y_pred = model.predict(X_test)
39 print(classification_report(y_test, y_pred, target_names=['Hợp lệ', 'Gian lận']))

```

```

38
39 # 3. LƯU MODEL
40 print("\nLưu model vào file 'fraud_model.pkl'...")
41 joblib.dump(model, 'fraud_model.pkl')

```

Chạy script để huấn luyện mô hình:

```
> python train_model.py
```

Bước 4: Lấy feature online và dự đoán

Bước cuối cùng mô phỏng một ứng dụng production. Khi có một yêu cầu dự đoán mới (ví dụ: một giao dịch vừa phát sinh), hệ thống chỉ cần biết ID của giao dịch đó.

Script `predict_online.py` sẽ sử dụng hàm `get_online_features` để truy vấn các giá trị đặc trưng mới nhất từ Online Store với độ trễ cực thấp. Vì các đặc trưng này được lấy từ cùng một định nghĩa đã dùng để huấn luyện, Feast đảm bảo tính nhất quán giữa hai môi trường, qua đó giải quyết triệt để vấn đề **Training-Serving Skew**. Một điểm quan trọng cần lưu ý là sau khi lấy được dữ liệu, chúng ta phải sắp xếp lại các cột đặc trưng để đảm bảo thứ tự của chúng khớp chính xác với dữ liệu đã dùng để huấn luyện mô hình.

predict_online.py

```

1 from feast import FeatureStore
2 import pandas as pd
3 import joblib
4
5 # 1. TẢI MODEL VÀ KẾT NỐI STORE
6 model = joblib.load('fraud_model.pkl')
7 store = FeatureStore(repo_path="feature_repo")
8
9 # 2. LẤY FEATURE ONLINE TỪ FEAST VỚI CÁC ID GIAO DỊCH MỚI
10 new_transaction_ids = ["284802", "284803", "284804", "284805", "284806"]
11 entity_rows = [{"transaction_id": tid} for tid in new_transaction_ids]
12
13 feature_names = [f"transaction_features_v1:V{i}" for i in range(1, 29)]
14 feature_names.append("transaction_features_v1:Amount")
15
16 print(f"Lấy online features cho các transaction IDs: {new_transaction_ids}...")
17 online_features = store.get_online_features(
18     features=feature_names,
19     entity_rows=entity_rows
20 ).to_dict()
21
22 # 3. CHUẨN BỊ DỮ LIỆU VÀ DỰ ĐOÁN
23 features_df = pd.DataFrame.from_dict(online_features)
24 feature_order = [f.split(":")[1] for f in feature_names]
25 X_predict = features_df[feature_order]

```



```

26 predictions = model.predict(X_predict)
27
28 # 4. HIỂN THỊ KẾT QUẢ
29 for i, tid in enumerate(new_transaction_ids):
30     prediction_label = "GIAN LẬN" if predictions[i] == 1 else "Hợp lệ"
31     print(f"Giao dịch ID {tid}: {prediction_label}")

```

Thực thi kịch bản dự đoán online:

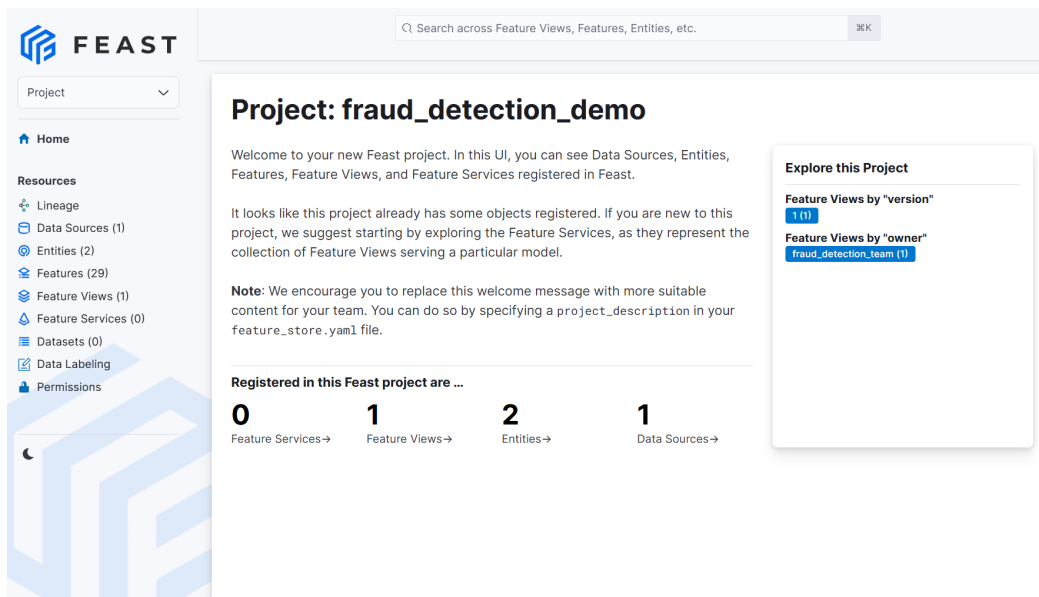
```
> python predict_online.py
```

Bước 5: Khám phá Feature Store với giao diện web

Feast cung cấp một giao diện web để trực quan hóa tất cả các thành phần đã đăng ký. Hãy chạy lệnh sau từ bên trong thư mục `feature_repo`:

```
> cd feature_repo
> feast ui
```

Sau đó, truy cập vào địa chỉ `http://127.0.0.1:8888` trên trình duyệt để xem các Entities, Feature Views đã tạo (Hình 11).



Hình 11: Giao diện web của Feast.

Ghi chú: Toàn bộ mã nguồn và tài nguyên của phần thực hành này có thể được tham khảo tại GitHub: https://github.com/duongtruongbinh/AIO2025_Tutorial_Feast.

IV. Câu hỏi trắc nghiệm

1. Đây là thách thức cốt lõi và phổ biến nhất mà Feature Store giải quyết, vốn xảy ra do sự khác biệt trong logic xử lý đặc trưng giữa môi trường huấn luyện và môi trường phục vụ?
 - (a) Data Leakage.
 - (b) Model Drift.
 - (c) Training-Serving Skew.
 - (d) Data Ingestion Bottleneck.
2. Trong kiến trúc của một Feature Store, thành phần nào chịu trách nhiệm lưu trữ dữ liệu lịch sử qua nhiều tháng hoặc năm để phục vụ cho việc huấn luyện mô hình?
 - (a) Online Store.
 - (b) Offline Store.
 - (c) Registry.
 - (d) Transformation Engine.
3. Tính năng nào của lớp Serving đảm bảo rằng khi tạo dữ liệu huấn luyện, giá trị đặc trưng được lấy đúng tại thời điểm trước khi sự kiện xảy ra, giúp ngăn chặn rò rỉ dữ liệu?
 - (a) Real-time Serving.
 - (b) Point-in-time Correctness.
 - (c) Batch Transformation.
 - (d) Data Monitoring.
4. Triết lý thiết kế cốt lõi của Feast là gì?
 - (a) Là một nền tảng “tất cả trong một” thay thế hoàn toàn Data Warehouse và các CSDL khác.
 - (b) Là một công cụ ETL chuyên biệt để biến đổi dữ liệu thô thành đặc trưng.
 - (c) Là một lớp điều phối mỏng, linh hoạt, được thiết kế để kết nối và tái sử dụng hạ tầng dữ liệu có sẵn.
 - (d) Là một công cụ điều phối workflow tương tự Airflow.
5. Trong Feast, đối tượng nào được dùng để định nghĩa một nhóm logic các đặc trưng và liên kết chúng với một Entity cùng một DataSource?
 - (a) Project.
 - (b) FeatureService.
 - (c) Entity.
 - (d) FeatureView.

6. Sau khi bạn định nghĩa các đối tượng (Entity, FeatureView...) trong file Python, bạn cần chạy lệnh CLI nào của Feast để đăng ký chúng vào registry?
 - (a) `feast init`.
 - (b) `feast materialize`.
 - (c) `feast apply`.
 - (d) `feast ui`.
7. Quá trình tính toán và nạp các giá trị đặc trưng mới nhất từ Offline Store vào Online Store được gọi là gì?
 - (a) Ingestion.
 - (b) Registration.
 - (c) Serving.
 - (d) Materialization.
8. Để lấy dữ liệu lịch sử nhằm tạo ra một bộ dữ liệu huấn luyện, bạn sẽ sử dụng hàm nào trong SDK của Feast?
 - (a) `store.get_online_features(...)`.
 - (b) `store.get_historical_features(...)`.
 - (c) `store.apply(...)`.
 - (d) `store.materialize(...)`.
9. Trong bài thực hành, tại sao script `prepare_data.py` lại cần thiết?
 - (a) Để tải dữ liệu từ Kaggle về máy.
 - (b) Để huấn luyện mô hình phát hiện gian lận.
 - (c) Vì bộ dữ liệu gốc thiếu hai cột quan trọng là `event_timestamp` và `transaction_id` mà Feast yêu cầu.
 - (d) Để nạp dữ liệu từ Offline Store vào Online Store.
10. Theo nội dung bài viết, Feast không được thiết kế để làm gì?
 - (a) Cung cấp một registry tập trung cho các định nghĩa đặc trưng.
 - (b) Thay thế hoàn toàn các công cụ điều phối workflow như Airflow.
 - (c) Phục vụ đặc trưng với độ trễ thấp cho các ứng dụng thời gian thực.
 - (d) Cung cấp một giao diện thống nhất để truy cập dữ liệu đặc trưng.

V. Tài liệu tham khảo

- [1] *Feast: The open source feature store — documentation*, <https://docs.feast.dev/>.
- [2] *Mlopsvn courses*, <https://courses-mlops-vn.vercel.app/>.
- [3] *What is a feature store?*, <https://feast.dev/blog/what-is-a-feature-store/>.

Phụ lục

1. **Datasets:** Các file dataset được đề cập trong bài có thể được tải tại [đây](#).
2. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải tại [đây](#).
3. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.

AIO_QAs-Verified

🔒 Private group · 1.4K members



Hình 12: Hình ảnh group facebook AIO Q&A.